GUÍA DE LABORATORIO 06

Comunicación entre procesos usando lenguaje C para Linux

Curso: Sistemas Operativos

Docente: Dra. Marcela Cruz Quispe

Integrantes:

Riomar Martinez Velasquez

Eduardo Josue Zela Quispe

Alexander Escarza Pacori

6 de octubre de 2026

Índice

1. Introducción **2**

2. Marco Teórico **2**
2.1. Comunicación entre Procesos . . . 2
2.2. Pipes . . . 2
2.3. Combinación con `fork()` . . . 2
2.4. Comunicación Bidireccional . . . 2

3. Ejercicio 1: Análisis del código **3**

4. Ejercicio 2: Análisis del código **4**
4.1. Descripción . . . 4
4.2. Código . . . 4
4.3. Captura de pantalla . . . 5

5. Ejercicio 3: Programa propio con un pipe **5**
5.1. Descripción . . . 5
5.2. Código . . . 5
5.3. Análisis del código . . . 6
5.4. Ejecución . . . 7
5.5. Captura de pantalla . . . 8
5.6. Limitación . . . 8

6. Ejercicio 4: Mejora con dos pipes **8**
6.1. Descripción . . . 8
6.2. Modificaciones realizadas . . . 8
6.3. Código . . . 8
6.4. Análisis del código . . . 10
6.5. Ejecución . . . 11
6.6. Captura de pantalla . . . 11
6.7. Conclusión . . . 11

7. Ejemplo breve de doble comunicación **11**
7.1. Ejemplo conceptual . . . 11
7.2. Ejemplo aplicado al código . . . 12
7.3. Flujo de mensajes . . . 12
7.4. Resultado . . . 12
7.5. Conclusión . . . 12

8. Conclusiones **12**

1. Introducción

El presente informe describe el desarrollo de la Guía de Laboratorio 06 del curso de Sistemas Operativos, cuyo tema central es la comunicación entre procesos utilizando el lenguaje C en Linux.

La comunicación entre procesos (IPC, por sus siglas en inglés) es un mecanismo fundamental que permite a dos o más procesos intercambiar información de forma segura. En Linux, uno de los mecanismos más utilizados son los *pipes* o tuberías, que permiten la transferencia de datos entre un proceso padre y un proceso hijo.

A lo largo de este informe se analizan los ejercicios propuestos en la guía, explicando el funcionamiento de los pipes, su creación mediante la función `pipe()`, su combinación con la función `fork()`, y la implementación de comunicación bidireccional mediante el uso de dos tuberías.

2. Marco Teórico

2.1. Comunicación entre Procesos

En Linux, cada proceso posee su propio espacio de memoria, lo que impide que un proceso acceda directamente a la memoria de otro. Para permitir el intercambio de datos entre procesos, el sistema operativo proporciona diversos mecanismos de comunicación, entre ellos los pipes.

2.2. Pipes

Un pipe es un canal de comunicación unidireccional. Se crea mediante la función `pipe()`, la cual devuelve dos descriptores de archivo:

- `descriptorTuberia[0]`: extremo de lectura.
- `descriptorTuberia[1]`: extremo de escritura.

La función `pipe()` retorna -1 en caso de error.

2.3. Combinación con `fork()`

Si se crea el pipe antes de llamar a `fork()`, tanto el proceso padre como el hijo obtienen una copia de los dos descriptores. Para evitar confusión y liberar recursos, cada proceso debe cerrar el extremo que no vaya a utilizar.

2.4. Comunicación Bidireccional

Un solo pipe es unidireccional. Para lograr comunicación en ambos sentidos, es necesario crear dos tuberías: una para la dirección padre $\rightarrow$ hijo, y otra para la dirección hijo $\rightarrow$ padre. Esto está documentado en la página 2 de la guía de laboratorio.

3. Ejercicio 1: Análisis del código

Este Código lo que ejecuta es la creación de un canal ente dos procesos, siendo que primero se declaran las acciones del Proceso hijo que deben esperar información que el Proceso Padre envíe, siendo en este caso Çiencia."

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    pid_t idProceso;
    int estadoHijo;
    int descriptorTuberia[2];
    char buffer[100];

    if (pipe(descriptorTuberia) == -1) {
        perror("No se puede crear Tuberia");
        exit(-1);
    }

    idProceso = fork();

    if (idProceso == -1) {
        perror("No se puede crear proceso");
        exit(-1);
    }

    if (idProceso == 0) {
        // Proceso Hijo
        close(descriptorTuberia[1]); // Cierra el extremo de escritura
        read(descriptorTuberia[0], buffer, 8);
        printf("Hijo: Recibido \"%s\"\n", buffer);
        exit(0);
    }

    if (idProceso > 0) {
        // Proceso Padre
        close(descriptorTuberia[0]); // Cierra el extremo de lectura
        printf("Padre: Envio \"Ciencia\"\n");
        strcpy(buffer, "Ciencia");
        write(descriptorTuberia[1], buffer, strlen(buffer) + 1);
        wait(&estadoHijo);
        exit(0);
    }

    return 1;
}
```

Listing 1: Comunicación entre procesos mediante tuberías (pipes) en C

4. Ejercicio 2: Análisis del código

4.1. Descripción

El código implementa una comunicación entre un proceso padre y un proceso hijo mediante una tubería o pipe en C , la comunicación se realiza del proceso hijo hacia el proceso padre.

4.2. Código

```c
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>

#define LEER 0
#define ESCRIBIR 1

char *frase = "Envia esto a traves de un tubo o pipe";

extern int errno;

int main() {
    int fd[2], bytesLeidos;
    char mensaje[100];
    int control;

    // Se crea la tuberia
    control = pipe(fd);

    if (control != 0) {
        perror("pipe: ");
        exit(errno);
    }

    // Se crea el proceso hijo
    control = fork();

    switch (control) {
        case -1:
            perror("fork: ");
            exit(errno);

        case 0:
            close(fd[LEER]);

            write(fd[ESCRIBIR], frase, strlen(frase) + 1);

            close(fd[ESCRIBIR]);

            exit(0);

        default:
```

```
45              close(fd[ESCRIBIR]);
46
47              bytesLeidos = read(fd[LEER], mensaje, 100);
48
49              printf("Leidos %d bytes: %s\n", bytesLeidos, mensaje);
50
51              close(fd[LEER]);
52      }
53
54      exit(0);
55  }
```

Listing 2: Comunicación entre procesos mediante una tubería (pipe) en C

4.3. Captura de pantalla

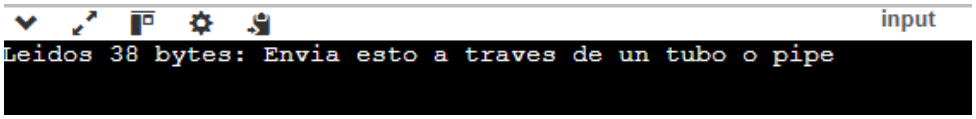


Figura 1: Ejecución del programa del Ejercicio 3

5. Ejercicio 3: Programa propio con un pipe

5.1. Descripción

Se elaboró un programa en lenguaje C que emplea un pipe para la comunicación entre un proceso padre y un proceso hijo. El padre envía un mensaje al hijo y el hijo lo recibe. Este ejercicio demuestra el funcionamiento básico de una tubería unidireccional.

5.2. Código

```
1  #include <sys/types.h>
2  #include <wait.h>
3  #include <unistd.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>
7
8  int main() {
9      pid_t idProceso;
10     int estadoHijo;
11     int tuberia[2];
12     char buffer[100];
13
14     if (pipe(tuberia) == -1) {
15         perror("No se puede crear Tuberia");
16         exit(-1);
17     }
18
```

```
19      idProceso = fork();
20
21      if (idProceso == -1) {
22          perror("No se puede crear proceso");
23          exit(-1);
24      }
25
26      if (idProceso == 0) {
27          close(tuberia[1]);
28
29          read(tuberia[0], buffer, 100);
30          printf("Hijo : Recibido \"%s\"\n", buffer);
31
32          close(tuberia[0]);
33          exit(0);
34      }
35
36      if (idProceso > 0) {
37          close(tuberia[0]);
38
39          strcpy(buffer, "Mensaje del padre");
40          printf("Padre: Envio    \"%s\"\n", buffer);
41          write(tuberia[1], buffer, strlen(buffer) + 1);
42
43          close(tuberia[1]);
44          wait(&estadoHijo);
45          exit(0);
46      }
47
48      return 1;
49  }
```

Listing 3: Programa del Ejercicio 3: comunicación unidireccional con un pipe

5.3. Análisis del código

Creación de la tubería

```
1  int tuberia[2];
2
3  if (pipe(tuberia) == -1) {
4      perror("No se puede crear Tuberia");
5      exit(-1);
6  }
```

Se crea la tubería con `pipe()`. Los descriptores `tuberia[0]` y `tuberia[1]` se usan para leer y escribir respectivamente. Si la función retorna -1, se muestra un error y el programa termina.

Creación del proceso hijo

```
1  idProceso = fork();
```

La función `fork()` crea el proceso hijo. Devuelve 0 en el hijo, un valor mayor que 0 en el padre, y -1 en caso de error.

Bloque del hijo

```
if (idProceso == 0) {
    close(tuberia[1]);

    read(tuberia[0], buffer, 100);
    printf("Hijo : Recibido \"%s\"\n", buffer);

    close(tuberia[0]);
    exit(0);
}
```

El hijo cierra `tuberia[1]` porque únicamente va a leer. Luego llama a `read()` y queda bloqueado hasta que el padre escriba. Al recibir el mensaje, lo imprime y termina con `exit(0)`.

Bloque del padre

```
if (idProceso > 0) {
    close(tuberia[0]);

    strcpy(buffer, "Mensaje del padre");
    printf("Padre: Envio    \"%s\"\n", buffer);
    write(tuberia[1], buffer, strlen(buffer) + 1);

    close(tuberia[1]);
    wait(&estadoHijo);
    exit(0);
}
```

El padre cierra `tuberia[0]` porque únicamente va a escribir. Copia el mensaje en el buffer, lo envía con `write()` y espera al hijo con `wait()` para evitar que quede huérfano.

5.4. Ejecución

```
gcc -o ejer3 ejer3.c
./ejer3
```

Listing 4: Compilación y ejecución del Ejercicio 3

```
Padre: Envio    "Mensaje del padre"
Hijo : Recibido "Mensaje del padre"
```

Listing 5: Salida del Ejercicio 3

5.5. Captura de pantalla

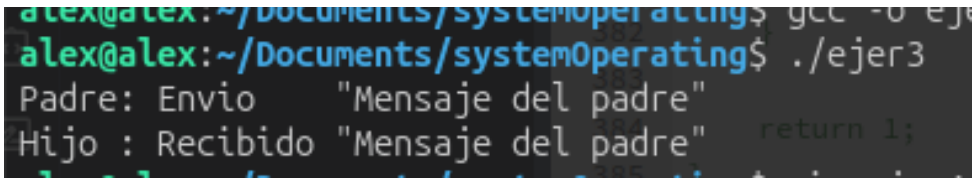


Figura 2: Ejecución del programa del Ejercicio 3

5.6. Limitación

El hijo no puede responder al padre porque un pipe es unidireccional. Los datos fluyen únicamente en una dirección.

6. Ejercicio 4: Mejora con dos pipes

6.1. Descripción

Se modificó el programa del Ejercicio 3 agregando un segundo pipe para permitir que el hijo también envíe datos al padre. Esta mejora está basada en la página 2 de la guía de laboratorio, donde se indica que para comunicación bidireccional es mejor abrir dos tuberías.

6.2. Modificaciones realizadas

Ejercicio 3	**Ejercicio 4**
`int tuberia[2];`	`int tuberiaPadreHijo[2];` `int tuberiaHijoPadre[2];`
Un `pipe()`	Dos `pipe()`
Hijo solo lee	Hijo lee y escribe
Padre solo escribe	Padre escribe y lee

Cuadro 1: Modificaciones del Ejercicio 3 al Ejercicio 4

6.3. Código

```c
#include <sys/types.h>
#include <wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    pid_t idProceso;
    int estadoHijo;
```

```
11      int tuberiaPadreHijo[2];
12      int tuberiaHijoPadre[2];
13      char buffer[100];
14
15      if (pipe(tuberiaPadreHijo) == -1) {
16          perror("No se puede crear Tuberia 1");
17          exit(-1);
18      }
19      if (pipe(tuberiaHijoPadre) == -1) {
20          perror("No se puede crear Tuberia 2");
21          exit(-1);
22      }
23
24      idProceso = fork();
25
26      if (idProceso == -1) {
27          perror("No se puede crear proceso");
28          exit(-1);
29      }
30
31      if (idProceso == 0) {
32          close(tuberiaPadreHijo[1]);
33          close(tuberiaHijoPadre[0]);
34
35          read(tuberiaPadreHijo[0], buffer, 100);
36          printf("Hijo : Recibido \"%s\"\n", buffer);
37
38          strcpy(buffer, "Hola padre");
39          printf("Hijo : Envio    \"%s\"\n", buffer);
40          write(tuberiaHijoPadre[1], buffer, strlen(buffer) + 1);
41
42          close(tuberiaPadreHijo[0]);
43          close(tuberiaHijoPadre[1]);
44          exit(0);
45      }
46
47      if (idProceso > 0) {
48          close(tuberiaPadreHijo[0]);
49          close(tuberiaHijoPadre[1]);
50
51          strcpy(buffer, "Hola hijo");
52          printf("Padre: Envio    \"%s\"\n", buffer);
53          write(tuberiaPadreHijo[1], buffer, strlen(buffer) + 1);
54
55          read(tuberiaHijoPadre[0], buffer, 100);
56          printf("Padre: Recibido \"%s\"\n", buffer);
57
58          close(tuberiaPadreHijo[1]);
59          close(tuberiaHijoPadre[0]);
60          wait(&estadoHijo);
61          exit(0);
62      }
63
64      return 1;
65  }
```

Listing 6: Programa del Ejercicio 4: comunicación bidireccional con dos pipes

6.4. Análisis del código

Creación de los dos pipes

```
int tuberiaPadreHijo[2];
int tuberiaHijoPadre[2];

pipe(tuberiaPadreHijo);
pipe(tuberiaHijoPadre);
```

Se crean dos tuberías: `tuberiaPadreHijo` para la dirección padre → hijo, y `tuberiaHijoPadre` para la dirección hijo → padre.

Bloque del hijo

```
if (idProceso == 0) {
    close(tuberiaPadreHijo[1]);
    close(tuberiaHijoPadre[0]);

    read(tuberiaPadreHijo[0], buffer, 100);
    printf("Hijo : Recibido \"%s\"\n", buffer);

    strcpy(buffer, "Hola padre");
    printf("Hijo : Envio    \"%s\"\n", buffer);
    write(tuberiaHijoPadre[1], buffer, strlen(buffer) + 1);

    close(tuberiaPadreHijo[0]);
    close(tuberiaHijoPadre[1]);
    exit(0);
}
```

El hijo cierra los extremos que no utiliza: no escribe en el pipe 1 y no lee del pipe 2. Ahora el hijo lee y escribe: recibe el mensaje del padre y responde por el segundo pipe.

Bloque del padre

```
if (idProceso > 0) {
    close(tuberiaPadreHijo[0]);
    close(tuberiaHijoPadre[1]);

    strcpy(buffer, "Hola hijo");
    printf("Padre: Envio    \"%s\"\n", buffer);
    write(tuberiaPadreHijo[1], buffer, strlen(buffer) + 1);

    read(tuberiaHijoPadre[0], buffer, 100);
    printf("Padre: Recibido \"%s\"\n", buffer);

    close(tuberiaPadreHijo[1]);
    close(tuberiaHijoPadre[0]);
    wait(&estadoHijo);
    exit(0);
}
```

El padre cierra los extremos opuestos: no lee del pipe 1 y no escribe en el pipe 2. Ahora el padre escribe y lee: envía el mensaje y espera la respuesta del hijo. El orden es: padre escribe, hijo lee, hijo escribe, padre lee.

6.5. Ejecución

```
gcc -o ejer4 ejer4.c
./ejer4
```

Listing 7: Compilación y ejecución del Ejercicio 4

```
Padre: Envio    "Hola hijo"
Hijo : Recibido "Hola hijo"
Hijo : Envio    "Hola padre"
Padre: Recibido "Hola padre"
```

Listing 8: Salida del Ejercicio 4

6.6. Captura de pantalla

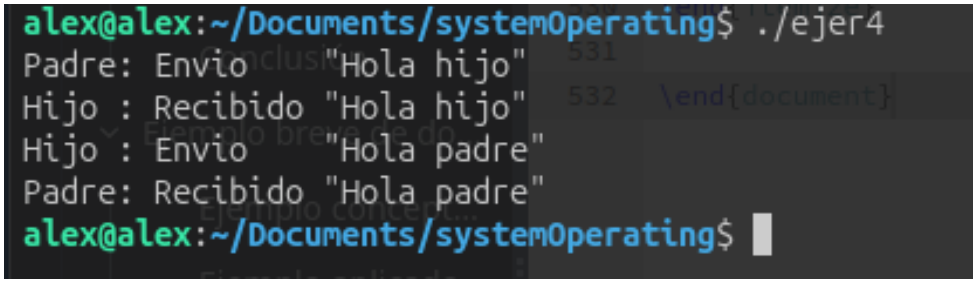


Figura 3: Ejecución del programa del Ejercicio 4

6.7. Conclusión

El Ejercicio 4 es una mejora del Ejercicio 3. Se agregó un segundo pipe para lograr comunicación bidireccional entre padre e hijo. La comunicación funciona correctamente en ambos sentidos.

7. Ejemplo breve de doble comunicación

7.1. Ejemplo conceptual

La doble comunicación entre procesos se puede comparar con una conversación telefónica:

- **Un solo pipe** = walkie-talkie: uno habla y el otro escucha. No pueden hablar al mismo tiempo.
- **Dos pipes** = teléfono: ambos pueden hablar y escuchar.

7.2. Ejemplo aplicado al código

Dirección	Pipe	Quién escribe	Quién lee
Padre → Hijo	`tuberiaPadreHijo`	Padre	Hijo
Hijo → Padre	`tuberiaHijoPadre`	Hijo	Padre

Cuadro 2: Flujo de comunicación en el Ejercicio 4

7.3. Flujo de mensajes

1. El padre escribe `"Hola hijo"` en `tuberiaPadreHijo`.
2. El hijo lee `"Hola hijo"` de `tuberiaPadreHijo`.
3. El hijo escribe `"Hola padre"` en `tuberiaHijoPadre`.
4. El padre lee `"Hola padre"` de `tuberiaHijoPadre`.

7.4. Resultado

```
Padre: Envio    "Hola hijo"
Hijo : Recibido "Hola hijo"
Hijo : Envio    "Hola padre"
Padre: Recibido "Hola padre"
```

Listing 9: Salida del ejemplo de doble comunicación

7.5. Conclusión

La doble comunicación se logra usando dos pipes: uno para cada dirección. Esto permite que ambos procesos envíen y reciban datos sin que se mezclen.

8. Conclusiones

- Un pipe es un mecanismo de comunicación unidireccional. Se crea con `pipe()` y proporciona dos descriptores: uno para lectura y otro para escritura.
- La función `fork()` permite crear un proceso hijo que hereda los descriptores del pipe. Es responsabilidad de cada proceso cerrar el extremo que no utiliza.
- Para lograr comunicación bidireccional entre procesos, es necesario utilizar dos pipes: uno para cada dirección.
- El uso de `wait()` permite que el proceso padre espere a que el hijo termine, evitando procesos huérfanos.
- La comunicación entre procesos mediante pipes es un mecanismo fundamental en Linux que permite el intercambio seguro de datos.